
Делегаты и события

Задание 1

Напишите программу, в которой с помощью делегатов и анонимных методов (лямбда-выражений) создается экземпляр делегата, который вызывается без аргументов, а результатом возвращает текстовое значение с названием дня недели ("Понедельник", "Вторник" и так до "Воскресенья"). При каждом новом вызове экземпляра результатом возвращается название следующего дня недели. После "Воскресенья" результатом возвращается "Понедельник" и так далее.

Задание 2

Напишите программу, содержащую статический метод с тремя действительными (тип **double**) аргументами. Результатом статического метода является ссылка на метод, вычисляющий квадратичный трехчлен с коэффициентами, определяемыми аргументами статического метода. Другими словами, если статический метод вызывается с числовыми аргументами **a**, **b** и **c**, то результатом является ссылка на метод, который для аргумента **x** типа **double** результатом вычисляет значение выражения $a \cdot x^2 + b \cdot x + c$.

Примеры использования

В следующем примере мы используем делегат для того, чтобы передавать ссылки на методы аргументом другому методу. Программа достаточно простая: описывается статический метод, который используется для того, чтобы создать таблицу значений другого метода. Таблица формируется так. Берется некоторый метод, у которого целочисленный аргумент и который возвращает целочисленный результат. Аргумент метода изменяется в заданных пределах. Для каждого значения аргумента отображается значение метода. Этот простой алгоритм не зависит от того, для какого именно метода строится таблица значений. Главное, чтобы у метода был целочисленный аргумент, и он должен возвращать целочисленный результат. Именно та ситуация, когда разумно использовать делегаты. Рассмотрим программный код, представленный ниже:

```
namespace pr170_1
{
    delegate int MyDelegate(int n); // Объявление делегата:

    class Program // Класс с главным методом:
    {
        // Статический метод для вычисления нечетных чисел:
        static int f(int n) {
            return 2 * n + 1;
        }
        // Статический метод для вычисления четных чисел:
        static int g(int n) {
            return 2 * n;
        }
        // Статический метод для вычисления квадратов чисел:
        static int h(int n) {
            return n * n;
        }
        // Статический метод, которому аргументом передается ссылка на метод:
        static void display(MyDelegate F, int a, int b) {
            Console.WriteLine("{0, -4}| {1, 4}", "x", "F(x)");
            Console.WriteLine("-----");
            for(int k = a; k <= b; k++) {
                Console.WriteLine("{0,-4}|{1,4}", k, F(k)); // Команда с вызовом экземпляра делегата
            }
            Console.WriteLine();
        }

        static void Main() // Главный метод
        {
            int a = 0, b = 5; // Диапазон изменения аргумента
            Console.WriteLine("Нечетные числа:");
            display(f, a, b); // Передача аргументом ссылки на метод
            Console.WriteLine("Четные числа:");

            display(g, a, b); // Передача аргументом ссылки на метод
        }
    }
}
```

```

        Console.WriteLine("Числа в квадрате:");
        display(h, a, b); // Передача аргументом ссылки на метод
        Console.ReadLine(); // Задержка
    }
}

```

В результате выполнения программы получаем следующее:

```

file:///C:/Users/.../Do...
Нечетные числа:
x | F(x)
---|---
0 | 1
1 | 3
2 | 5
3 | 7
4 | 9
5 | 11

Четные числа:
x | F(x)
---|---
0 | 0
1 | 2
2 | 4
3 | 6
4 | 8
5 | 10

Числа в квадрате:
x | F(x)
---|---
0 | 0
1 | 1
2 | 4
3 | 9
4 | 16
5 | 25

```

Программа небольшая и простая. В программе командой

```
delegate int MyDelegate(int n);
```

объявляется делегат, экземпляр которого может ссылаться на метод, имеющий целочисленный аргумент и возвращающий целочисленный результат.

В классе с главным методом программы описано еще несколько статических методов. Методы **f()**, **g()** и **h()** соответствуют параметрам делегата **MyDelegate**: у каждого из этих методов один целочисленный аргумент, и каждый метод результатом возвращает целое число.

При заданном целочисленном аргументе **n** метод **f()** возвращает значение **2*n+1** (нечетное число), метод **g()** возвращает значение **2*n** (четное число), а метод **h()** возвращает значение **n*n** (число в квадрате). Еще один статический метод **display()** описан с тремя аргументами.

Первый аргумент **F** имеет тип **MyDelegate** - то есть это переменная, которая может ссылаться на экземпляр делегата. Мы уже знаем, что такой переменной можно присваивать ссылку на метод. Поэтому при вызове метода первым аргументом можно передавать ссылку на метод (с целочисленным аргументом и целочисленным результатом). Другими словами, под аргументом **F** мы можем подразумевать название метода, который можно вызывать, передав ему целочисленный аргумент. Еще два целочисленных аргумента (**a** и **b**) определяют диапазон, на котором табулируется переданный первым аргументом метод. Табуляция выполняется просто: с помощью цикла в консоль выводится значение аргумента **k** и значение метода **F(k)**.

В командах

```
Console.WriteLine("{0, -4}| {1, 4}", "x", "F(x)");
```

и

```
Console.WriteLine("{0,-4}|{1,4}", k, F(k));
```

инструкция {0, -4} означает, что под отображаемое значение выделяется 4 позиции и выравнивание выполняется по левому краю. Инструкция {1, 4} означает, что под отображаемое значение выделяется 4 позиции и выравнивание выполняется по правому краю.

*В главном методе программы при заданных значениях переменных **a** и **b** командами*

```
display(f, a, b);
```

```
display(g, a, b);
```

```
display(h, a, b);
```

*табулируются методы **f()**, **g()** и **h()** соответственно. Чтобы передать метод аргументом методу **display()**, достаточно указать имя метода первым аргументом метода **display()**. Например, при выполнении команды*

```
display(f, a, b);
```

*выполняется код метода **display()**, но только вместо аргумента **F** в программном коде следует подразумевать **f**. То же касается и других команд с вызовом **display()**.*

Еще один пример, который мы рассмотрим, касается ситуации, когда с помощью делегатов мы создаем иллюзию (хотя это и не совсем иллюзия), что результатом выражения является метод. Рассмотрим программный код, представленный в примере ниже:

```
namespace pr171_1
{
    delegate int Method(int n); // Объявление делегата
    class MyClass { // Класс с индекатором
        private int num; // Закрытое целочисленное поле
        public MyClass(int n) { // Конструктор с целочисленным аргументом
            num = n;
        }
        private int first(int n) { // Первый закрытый метод
            return n + num;
        }
        private int second(int n) { // Второй закрытый метод
            return n - num;
        }
        private int third(int n) { // Третий закрытый метод
            return n * num;
        }
        // Индексатор с целочисленным индексом. Результат - ссылка на экземпляр
        // делегата:
        public Method this[int k] {
            get { // Аксессор для считывания значения
                switch(k) {
                    case 0:
                        return first;
                    case 1:
                        return second;
                    default:
                        return third;
                }
            }
        }
    }
}
```

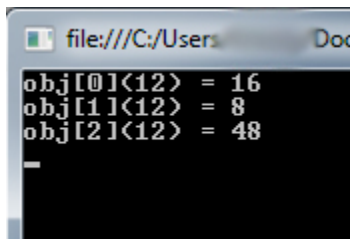
```

class Program // Класс с главным методом
{
    static void Main() // Главный метод

    {
        int x = 12; // Переменная
        MyClass obj = new MyClass(4); // Создание объекта
        for (int k = 0; k <= 2; k++) // Индексирование объекта
        {
            Console.WriteLine("obj[{0}][{1}] = {2}", k, x, obj[k](x));
        }
        Console.ReadLine(); // Задержка
    }
}

```

При выполнении программы получим следующий результат:



Делегат **Method** объявляется командой

```

delegate int Method(int n);

```

и соответствует методам с одним целочисленным аргументом и целочисленным результатом.

В классе **MyClass** мы описываем закрытое целочисленное поле **num**, значение которому присваивается при создании объекта. Еще в классе описаны три закрытых метода.

Метод **first()** возвращает в качестве результата сумму значения своего аргумента и поля **num**.

Метод **second()** результатом возвращает разность аргумента и поля **num**.

Метод **third()** возвращает результатом произведение значений аргумента и поля **num**. Еще раз подчеркнем, что все три метода закрытые, поэтому прямого доступа к ним вне пределов класса нет. Но в классе описан индексатор с **get**-аксессором. Тип результата, возвращаемого аксессором, указан как **Method**.

Это означает, что при индексировании объекта мы результатом получаем ссылку на экземпляр делегата **Method**. Но фактически это ссылка на метод, который соответствует параметрам делегата **Method**.

Индексатор описан так, что если индекс равен 0, то результатом возвращается значение **first** (ссылка на метод **first()** индексируемого объекта). Если индекс равен 1, то результатом возвращается значение **second** (ссылка на метод **second()** индексируемого объекта). При всех других значениях индекса результатом возвращается ссылка **third** на метод **third()** индексируемого объекта.

*В описании аксессуара указано, что результатом он возвращает значение типа **Method**. Что это означает? Это значит, что результатом аксессуара является ссылка на экземпляр делегата **Method**. При вызове аксессуара для записи результата выделяется переменная (типа **Method**). Эта переменная может ссылаться на экземпляр делегата. Фактическим значением аксессуар возвращает ссылку на метод. Получается, что переменной типа **Method** присваивается ссылка на метод. В этом случае, как мы помним, автоматически создается экземпляр делегата, который ссылается на метод. Ссылка на экземпляр делегата записывается в переменную, выделенную для запоминания результата аксессуара.*

В главном методе программы мы проверяем работу индексатора. Для этого командой

```
MyClass obj = new MyClass(4);
```

создаем объект **obj** класса **MyClass**. Целочисленное поле **num** этого объекта равно 4 (аргумент конструктора). Далее для переменной **x** со значением 12 в цикле при фиксированном значении переменной **k** (изменяется в пределах от 0 до 2 включительно) вычисляется значение выражения **obj[k](x)**. Несложно заметить, что в этом случае с аргументом **x** из объекта **obj** последовательно вызываются методы **first()**, **second()** и **third()**.

Таким образом, выражения вида **obj[k]** мы можем отождествлять с именем метода.

Лямбда-выражения

Анонимный метод может возвращаться в качестве результата другого метода. В примере ниже это проиллюстрировано:

```
namespace pr176_1
```

```
{
```

```
    delegate int MyDelegate(); // Объявление делегата:
```

```
    class Program // Класс с главным методом:
```

```
    {
```

```
        // Статический метод результатом возвращает ссылку на экземпляр делегата:
```

```
        static MyDelegate calculate(int n) {
```

```
            int count = 0; // Локальная переменная:
```

```
            // Результат реализован через анонимный метод:
```

```
            return delegate() {
```

```
                count += n;
```

```
                return count;
```

```
            };
```

```
        }
```

```
        static void Main() // Главный метод:
```

```
        {
```

```
            MyDelegate next = calculate(1); // Переменная типа делегата:
```

```
            for (int i = 1; i <= 5; i++)
```

```
            {
```

```
                Console.Write(next() + " "); // Вызов экземпляра делегата:
```

```
            }
```

```
            Console.WriteLine();
```

```
            // Новое значение переменной типа делегата:
```

```
            next = calculate(3);
```

```
            for (int i = 1; i <= 5; i++)
```

```
            {
```

```
                // Вызов экземпляра делегата:
```

```
                Console.Write(next() + " ");
```

```
            }
```

```
            Console.WriteLine();
```

```
            // Задержка:
```

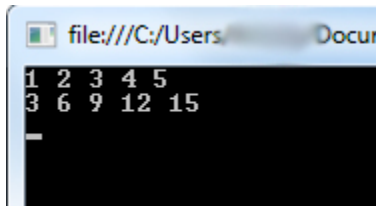
```
            Console.ReadLine();
```

```
        }
```

```
    }
```

```
}
```

При выполнении программы получаем такой результат:



Делегат **MyDelegate** описан для создания экземпляров, которые могут ссылаться на методы без аргументов и возвращающие целочисленный результат. Еще мы описываем статический метод **calculate()**, у которого целочисленный аргумент (обозначен как **n**) и который результатом возвращает ссылку на экземпляр делегата **MyDelegate**. В теле метода объявлена локальная целочисленная переменная **count** с начальным нулевым значением. Далее следует **return**-инструкция и возвращаемым значением указано следующее выражение (после этого выражения в коде метода стоит точка с запятой):

```
return delegate () {  
  
    count += n;  
  
    return count;  
  
};
```

Это код анонимного метода без аргументов, который результатом возвращает целое число. Этот анонимный метод возвращается результатом метода **calculate()**.

Тип результата для метода **calculate()** указан как **MyDelegate**. Формально это ссылка на экземпляр делегата **MyDelegate**. При вызове метода **calculate()** под результат метода выделяется переменная типа делегата **MyDelegate**. Результатом метод **calculate()** возвращает анонимный метод, который присваивается переменной, выделенной для записи результата. В итоге создается экземпляр делегата, который ссылается на анонимный метод. Ссылка на этот экземпляр записывается в переменную, выделенную для запоминания результата метода **calculate()**. Поэтому результатом метода **calculate()** является ссылка на экземпляр делегата **MyDelegate**, а этот экземпляр ссылается на упомянутый выше анонимный метод.

В главном методе программы командой

```
MyDelegate next = calculate(1);
```

объявляется переменная **next** типа делегата **MyDelegate**, и значением ей присваивается результат вызова метода **calculate()** с аргументом 1. Затем с помощью конструкции цикла

несколько раз подряд вычисляется значение выражения **next()**. И каждый раз мы получаем новое значение (числа 1, 2, 3 и так далее). Почему так происходит? При вызове метода **calculate()** с аргументом 1 создается локальная переменная **count** с начальным значением 0. Мы уже знаем, что метод **calculate()** возвращает результатом анонимный метод, который при вызове каждый раз увеличивает значение переменной **count** (в данном случае на 1) и это новое значение возвращает как результат. В итоге получается так: переменная **next** ссылается на экземпляр делегата, который ссылается на анонимный метод, а этот метод содержит ссылку на переменную **count**, созданную при вызове метода **calculate()**. После завершения вызова метода **calculate()** локальная переменная **count** должна бы удаляться. Но поскольку на эту переменную есть ссылка в анонимном методе, то она "выживает". После первого вызова анонимного метода через переменную **next** значение переменной **count** становится равным 1. При втором вызове она становится равной 2 и так далее. Поэтому при вычислении значения выражения **next()** мы каждый раз получаем новое число (на единицу больше предыдущего).

После выполнения команды

```
next = calculate(3);
```

в переменную **next** записывается ссылка на новый экземпляр делегата, ссылающийся на новый анонимный метод, который ссылается на новую локальную переменную **count**. Она получила начальное значение 0 при вызове метода **calculate()**. И теперь каждый раз при вычислении значения выражения **next()** мы получаем новое число, которое больше предыдущего на 3.